IBM Innovation Center for Education

NCS Nihon Communication Solutions

THE YENEPOYA INSTITUTE OF ARTS SCIENCE COMMERCE AND MANAGEMENT

PROJECT-LOW LEVEL DESIGN

On

Behavioral Analytics for Energy Security

NAME : Adarsh A

REG NO : 23BSCED007

GUIDED BY:

TABLE OF CONTENT :

1. Introduction 1
1.1. Purpose and Scope 2
1.2. Project Executive Summary 2
1.3. Constraints 2
1.4. Document Organization 3
2. System Overview 3
3. Module Design 4
3.1. Data Collection Module 4
3.2. Feature Engineering Module 4
3.3. Anomaly Detection Module 4
3.4. Reporting Module 5
3.5. Output and Export Module 5
4. Data Flow Design 5
5. Database Design 5
5.1. Login Records Table 5
5.2. User Statistics Table 6
5.3. Anomaly Alert Table 6
6. User Interface Design 6
7. Security Implementation 6
8. Testing and Validation 7
9. Project References 7
10. Glossary 7
11. Front-end (Notebook Output) 8
12. Back-end (Python Code) 9

1. INTRODUCTION

1.1 Purpose and Scope

Purpose:

This document provides a detailed low-level design of a Behavioral Analytics system tailored for the Energy Security sector. The goal is to design a system that analyzes user login patterns from an energy security platform, calculates average login hours, and flags suspicious logins occurring at 3 AM as potential security threats.

The system focuses on demonstrating how user behavior can be analyzed using Python and Pandas to detect anomalies — a concept central to cybersecurity monitoring of critical infrastructure.

Scope — The system covers:

- User login data ingestion and preprocessing using Pandas
- Feature engineering — extraction of login hour from timestamp
- Statistical heuristic-based anomaly detection (3 AM login flagging)
- User behavior profiling with average login hour calculation
- Export of anomaly report and full security audit log as CSV/XLS

In Scope:

- Loading and parsing user_login_data.csv with Pandas
- Extraction of login_hour from login_time timestamps
- Calculation of avg_login_hour, total_logins, count_3am per user
- Flagging any login at hour == 3 as SUSPICIOUS with HIGH risk
- Generating anomaly_report.csv and full_security_audit_log.csv

Out of Scope:

- Real-time network monitoring or packet inspection
- Machine learning-based anomaly detection models
- Integration with SIEM or external security platforms
- Web dashboard or GUI interface

1.2 Project Executive Summary

Energy security systems protect critical infrastructure such as power grids, substations, and SCADA systems from unauthorized access and cyber threats. Behavioral analytics monitors how and when authorized users access these systems and identifies patterns that deviate from normal behavior.

This project introduces a structured behavioral analytics pipeline that:

- Loads a CSV dataset of user login events from an energy security platform
- Extracts the login hour from timestamps for each event
- Calculates per-user statistics — average login hour, total logins, and 3 AM login count
- Flags any login occurring at 3 AM as a high-risk suspicious event
- Separates users into flagged (HIGH RISK) and safe categories
- Exports an anomaly report and a full security audit log

The system is implemented using:

- **Core Analysis:** Python, Pandas

- **Environment:** Jupyter Notebook
- **Storage:** CSV / XLS Files (anomaly_report, full_security_audit_log)
- **Visualization:** Pandas DataFrames, Console Output

1.3 Constraints

Technical Constraints:

- Analysis is based on static CSV data — no real-time data stream processing
- The 3 AM rule is a simplified heuristic — not a machine learning model
- Jupyter Notebook requires Python 3.x and Pandas to be installed
- Large datasets may require chunked reading for memory efficiency

Security Constraints:

- System must not expose raw login data to unauthorized viewers
- Exported CSV report files must be stored securely
- Analysis runs locally — no external API calls or cloud transmission

Operational Constraints:

- Designed for academic/demo purposes
- Local execution only — no public cloud or network deployment
- Requires Python 3.x, Pandas, and Jupyter Notebook to be installed

1.4 Document Organization

The document is structured into:

1. Introduction
2. System Overview
3. Module Design
4. Data Flow
5. Database Structure
6. Interface Design
7. Security Implementation
8. Testing Strategy

2. System Overview

The system is designed using a **Pandas-based data analysis pipeline**:

- Login records are loaded from a CSV file into a Pandas DataFrame
- The login_hour is extracted from each timestamp using .dt.hour
- User-level statistics are computed via groupby() and aggregation
- Any login at hour == 3 is flagged as SUSPICIOUS with HIGH risk level
- Results are exported as a structured anomaly report and audit log

The system ensures every login event is evaluated against the 3 AM heuristic rule, making the analysis transparent and reproducible while mimicking real-world energy grid security monitoring workflows.

3. Module Design

3.1 Data Collection Module

- Loads user_login_data.csv using pd.read_csv() with parse_dates=['login_time']

- Validates dataset structure — checks columns, shape, and null values
- Displays first 10 records, unique user count, and date range for verification

3.2 Feature Engineering Module

- Extracts login_hour (0–23) from login_time using .dt.hour
- Extracts login_date for date-level analysis
- Prints login hour distribution to understand normal vs. off-hours access patterns

3.3 Anomaly Detection Module

- Groups data by user_id, username, and department using groupby()
- Calculates total_logins, avg_login_hour, min_hour, max_hour, count_3am per user
- Applies boolean mask: df['is_suspicious'] = df['login_hour'] == 3
- Generates anomaly summary: total logins, normal logins, suspicious logins, anomaly rate

3.4 Reporting Module

- Adds flag_reason ('Unauthorized 3 AM access to Energy Security System') and risk_level ('HIGH')
- Identifies flagged users (count_3am > 0) vs. safe users for security investigation
- Builds complete security audit log with status and risk_level columns applied to all rows

3.5 Output and Export Module

- Exports suspicious logins to anomaly_report.csv for security team action
- Exports full annotated dataset to full_security_audit_log.csv for record keeping
- Prints export confirmation with total flagged events and total audit record count

4. Data Flow Design

1. Load user_login_data.csv into Pandas DataFrame with parsed datetime
2. Extract login_hour and login_date from login_time column
3. Compute user-level statistics: avg_login_hour, total_logins, count_3am
4. Apply anomaly rule: flag all rows where login_hour == 3 as SUSPICIOUS
5. Separate flagged users from safe users and print anomaly report summary
6. Build full security audit log with status and risk_level applied to all rows
7. Export anomaly_report.csv and full_security_audit_log.csv
8. Security analyst reviews exported reports for further investigation

5. Database Design

Storage Used: CSV / XLS Files (anomaly_report.xls, full_security_audit_log.xls)

5.1 Login Records Table

Field	Data Type	Description
user_id	String	Unique identifier for each user
username	String	Display name of the user
department	String	Department the user belongs to
login_time	DateTime	Full timestamp of the login event

Field	Data Type	Description
login_hour	Integer	Hour (0–23) extracted from login_time
login_date	Date	Date extracted from login_time
location	String	Location from which login was made

5.2 User Statistics Table

Field	Data Type	Description
user_id	String	Unique user identifier
username	String	Name of the user
department	String	User department
total_logins	Integer	Total number of login events
avg_login_hour	Float	Mean login hour across all sessions
min_hour	Integer	Earliest login hour recorded
max_hour	Integer	Latest login hour recorded
count_3am	Integer	Number of logins at 3 AM
flagged	Boolean	True if count_3am > 0

5.3 Anomaly Alert Table

Field	Data Type	Description
user_id	String	User identifier
username	String	User name
department	String	Department
login_time	DateTime	Timestamp of suspicious login
login_hour	Integer	Hour of suspicious login (always 3)
location	String	Login location
flag_reason	String	Unauthorized 3 AM access to Energy Security System
risk_level	String	HIGH

6. User Interface Design

The system includes:

- Jupyter Notebook with step-by-step markdown explanations and code cells
- Console anomaly report printout: total logins, normal vs. suspicious count, anomaly rate
- Pandas DataFrame display of flagged users and safe users
- Complete security audit log table with status and risk level columns
- Exported XLS/CSV reports viewable in Excel or any spreadsheet tool

The notebook interface is designed for clarity and academic presentation — each step is documented with markdown headers, rationale, and output tables.

7. Security Implementation

- Anomaly heuristic (login_hour == 3) is purely analytical — no real server exposed
- Exported report files are stored locally and not transmitted externally
- Analysis runs in a sandboxed Jupyter Notebook environment
- No raw credentials or passwords are processed — only login timestamps
- Each user's behavior is independently evaluated to prevent cross-contamination

8. Testing and Validation

- Functional testing of CSV loading and datetime parsing
- Login hour extraction validation — confirmed via value_counts() output
- Anomaly detection accuracy testing — verified count_3am matches flagged records
- Export validation — confirmed anomaly_report.csv and audit log row counts match expected values

9. Project References

- Python Pandas Official Documentation — pandas.pydata.org
- Jupyter Notebook Official Documentation — jupyter.org
- OWASP Behavioral Analytics and Anomaly Detection Guidelines
- SCADA Cybersecurity Best Practices — ICS-CERT

10. Glossary

Behavioral Analytics: Monitoring user behavior patterns to detect deviations or threats

Anomaly Detection: Identification of data points that deviate from expected patterns

SCADA: Supervisory Control and Data Acquisition — industrial control system for energy infrastructure

Pandas: Python data analysis library used for tabular data manipulation

Statistical Heuristic: A rule-based approach for identifying suspicious events without machine learning

Login Hour: The hour of day (0–23) when a user logged into the energy security system

is_suspicious: Boolean column: True when login_hour == 3

count_3am: Number of times a user logged in at exactly 3 AM

11. FRONT-END (Jupyter Notebook Output)

The Jupyter Notebook serves as the front-end. Below are representative outputs from each key step:

Step 2 — Dataset Overview (Sample Output)

user_id	username	department	login_time	location
U001	Alice Johnson	Grid Operations	2024-01-15 08:32:00	Control Room A
U002	Bob Smith	SCADA Monitoring	2024-01-15 09:15:00	Remote Station 1
U003	Carol White	Cyber Security	2024-01-15 03:14:00	HQ
U004	David Lee	Grid Operations	2024-01-15 10:45:00	Control Room B
U005	Eve Turner	Maintenance	2024-01-15 11:20:00	Field Office

Step 5 — User Statistics DataFrame

username	department	total_logins	avg_login_hour	count_3am	flagged
Alice Johnson	Grid Operations	12	9.25	0	False
Bob Smith	SCADA Monitoring	9	10.11	0	False
Carol White	Cyber Security	15	7.80	3	True
David Lee	Grid Operations	11	11.36	0	False
Eve Turner	Maintenance	8	8.50	1	True
Frank Ray	IT Security	14	9.00	0	False

Step 6 — Anomaly Report Summary

Metric	Value
Total Login Events	69
Normal Logins	65
Suspicious (3 AM)	4
Anomaly Rate	5.8%
Unique Users	6
Flagged Users	2

Step 7 — Flagged Users (HIGH RISK)

user_id	username	department	count_3am	avg_login_hour
U003	Carol White	Cyber Security	3	7.80
U005	Eve Turner	Maintenance	1	8.50

12. BACK-END – PYTHON

The back-end is implemented in Python using Pandas inside a Jupyter Notebook. Key implementation steps:

Key Code Snippets

Step	Operation	Python Code
1	Import	`import pandas as pd`
2	Load Data	`df = pd.read_csv('user_login_data.csv', parse_dates=['login_time'])`
3	Explore	`df.info() | df.isnull().sum()`
4	Feature Eng.	`df['login_hour'] = df['login_time'].dt.hour`
5	User Stats	`user_stats = df.groupby(['user_id','username','department']).agg(...)`
6	Flag Anomalies	`df['is_suspicious'] = df['login_hour'] == 3`
7	Separate Users	`flagged = user_stats[user_stats['count_3am'] > 0]`
8	Audit Log	`df['status'] = df['is_suspicious'].apply(lambda x: 'SUSPICIOUS' if x else 'Normal')`
9	Export	`df.to_csv('full_security_audit_log.csv', index=False)`

Step 8 — Security Audit Log (Sample Output)

username	department	login_time	login_hour	status	risk_level
Alice Johnson	Grid Operations	2024-01-15 08:32:00	8	✓ Normal	LOW
Carol White	Cyber Security	2024-01-16 03:14:00	3	■ SUSPICIOUS	HIGH
Bob Smith	SCADA Monitoring	2024-01-15 09:15:00	9	✓ Normal	LOW
Eve Turner	Maintenance	2024-01-17 03:07:00	3	■ SUSPICIOUS	HIGH
David Lee	Grid Operations	2024-01-15 10:45:00	10	✓ Normal	LOW